Data Patterns and Representations: Final Project
03/31/2026
Group 9: Sadia Rahman | Brooke Rice

Representation-Specific Analysis

Data Representation and Preprocessing

In [1]:
```
import pandas as pd
import numpy as np
```

In [2]:
```
# load data
repatriations = pd.read_excel("KHSM Repatriations fy25m11.xlsx", sheet_name = None)
enc_2020_2023 = pd.read_csv("nationwide-encounters-fy20-fy23-aor.csv")
enc_2023_2026 = pd.read_csv("nationwide-encounters-fy23-fy26-jan-aor.csv")
policy = pd.read_csv("policy_markers_monthly_2015_2025.csv")
```

The raw datasets were loaded from multiple sources, including DHS repatriation data, CBP encounter data, and a custom policy marker dataset. The repatriation data came from a multisheet Excel file, while the encounter data were split across two Excel files covering different fiscal year ranges. The policy dataset was loaded from a CSV file containing monthly policy indicators. All datasets were imported into pandas DataFrames so they could be cleaned and processed consistently across sources.

Clean DHS Monthly Repatriations

In [3]:
```
# extract the relevant sheet from the excel file
dhs_monthly = repatriations["Monthly Repatriations"].copy()

# standardize column names (remove newline characters and strip extra whitespace)
dhs_monthly.columns = (dhs_monthly.columns.astype(str)
                       .str.replace("\n", " ", regex = False)
                       .str.strip()
                      )

# rename columns for clarity and consistency
dhs_monthly = dhs_monthly.rename(columns = {"Fiscal Year": "year", # 'Fiscal Year'
                                            "Month": "month_label", # 'Month' to ra
                                            "Quantity": "repatriations" # 'Quantity
                                           })

dhs_monthly["year"] = pd.to_numeric(dhs_monthly["year"], errors = "coerce") # conve
dhs_monthly["month_num"] = dhs_monthly["month_label"].str[:2].astype(int) # extract
# convert fiscal month index to calendar date
dhs_monthly["month_num"] = np.where(dhs_monthly["month_num"].isin([1, 2, 3]),
                                    dhs_monthly["month_num"] + 9,
```

```
                                        dhs_monthly["month_num"] - 3
                                        )
# convert fiscal year to calendar date
dhs_monthly["calendar_year"] = np.where(dhs_monthly["month_num"].isin([10, 11, 12])
                                        dhs_monthly["year"] - 1,
                                        dhs_monthly["year"]
                                        )
# create a standardized monthly date column
dhs_monthly["date"] = pd.to_datetime(
    dict(year = dhs_monthly["calendar_year"], month = dhs_monthly["month_num"], day
)
dhs_monthly = dhs_monthly[["date", "year", "month_num", "repatriations"]].dropna()

# validation

print("DHS Monthly Shape:", dhs_monthly.shape) # check dataset shape
print("\nSample rows:")
print(dhs_monthly.head()) # preview cleaned data
print("\nMissing values per column:")
print(dhs_monthly.isna().sum()) # check for missing values
print("\nDuplicate dates:", dhs_monthly["date"].duplicated().sum()) # check for dup
print("\nDate range:", dhs_monthly["date"].min(), "to", dhs_monthly["date"].max())
print("\nRepatriations summary stats:")
print(dhs_monthly["repatriations"].describe()) # basic sanity check on repatriation
# check for missing months in the sequence
expected_range = pd.date_range(start = dhs_monthly["date"].min(),
                               end = dhs_monthly["date"].max(),
                               freq = "MS")
missing_months = expected_range.difference(dhs_monthly["date"])
print("\nMissing months in sequence:", len(missing_months))
print(missing_months[:5]) # show first few if any
```

```
DHS Monthly Shape: (182, 4)

Sample rows:
        date  year  month_num  repatriations
0 2009-10-01  2010         10          79200
1 2009-11-01  2010         11          69680
2 2009-12-01  2010         12          63560
3 2010-01-01  2010          1          69200
4 2010-02-01  2010          2          75750

Missing values per column:
date             0
year             0
month_num        0
repatriations    0
dtype: int64

Duplicate dates: 0

Date range: 2009-10-01 00:00:00 to 2024-11-01 00:00:00

Repatriations summary stats:
count       182.000000
mean      61840.714286
std       29037.522287
min       27080.000000
25%       40067.500000
50%       51780.000000
75%       71817.500000
max      147080.000000
Name: repatriations, dtype: float64

Missing months in sequence: 0
DatetimeIndex([], dtype='datetime64[ns]', freq='MS')
```

The DHS repatriation dataset was cleaned and transformed into a standardized monthly time series. Column names were first normalized to remove formatting inconsistencies, and key variables were renamed for clarity. The fiscal year and month fields were converted into numeric formats, and a monthly date variable was created to align with the other datasets. The cleaned dataset contains 182 monthly observations spanning from January 2010 to February 2025. We checked for missing values and duplicate dates and found none, confirming that the dataset is consistently structured at a monthly level. We also verified that there were no gaps in the timeline, so the time series has complete coverage. Summary statistics show that repatriation counts average around 61,841, with values ranging from 27,080 to 147,080. This variation over time suggests that there are meaningful changes in enforcement activity, which supports further analysis of trends and policy effects. Overall, the DHS dataset is clean, consistent, and ready to be merged with the other sources.

Combine and Clean CBP encounters

In [4]:

```
# preview original file sizes before cleaning
print("Original file shapes:")
print("enc_2020_2023:", enc_2020_2023.shape)
print("enc_2023_2026:", enc_2023_2026.shape)

# standardize and combine files
enc_2023_2026["Fiscal Year"] = enc_2023_2026["Fiscal Year"].apply(lambda x: 2026 if
enc_2020_2023 = enc_2020_2023[enc_2020_2023["Fiscal Year"] != 2023] # remove FY2023
encounters = pd.concat([enc_2020_2023, enc_2023_2026], ignore_index = True) # combi

# confirm the combined dataset size
print("\nAfter removing FY2023 overlap and combining:")
print("Combined encounters shape:", encounters.shape)

# check which fiscal years are present after combining
print("\nFiscal years present in combined file:")
print(sorted(encounters["Fiscal Year"].dropna().unique()))

# clean and standardize columns
# rename columns for easier use throughout the notebook
encounters = encounters.rename(columns = {"Fiscal Year": "year",
                                          "Month (abbv)": "month_abbr",
                                          "Encounter Count": "encounters",
                                          "Title of Authority": "title_authority"
                                          })

# convert month abbreviations into numeric month values for building a proper month
month_map = {"OCT": 10, "NOV": 11, "DEC": 12,
             "JAN": 1, "FEB": 2, "MAR": 3,
             "APR": 4, "MAY": 5, "JUN": 6,
             "JUL": 7, "AUG": 8, "SEP": 9
             }

# clean month abbreviations before mapping
encounters["month_abbr"] = encounters["month_abbr"].str.upper().str[:3]
encounters["month_num"] = encounters["month_abbr"].map(month_map)

# check whether any month abbreviations failed to map
print("\nMissing month_num values:", encounters["month_num"].isna().sum())

# convert fiscal year to calendar date
encounters["calendar_year"] = np.where(encounters["month_num"].isin([10, 11, 12]),
                                       encounters["year"] - 1,
                                       encounters["year"]
                                       )

# create a standardized monthly date column
encounters["date"] = pd.to_datetime(dict(year = encounters["calendar_year"], month

# preview cleaned encounter rows
print("\nSample cleaned encounter rows:")
print(encounters[["year", "month_abbr", "month_num", "calendar_year", "date", "enco

# check for missing values in key fields
print("\nMissing values in key columns:")
```

```
print(encounters[["year", "month_abbr", "month_num", "calendar_year", "date", "enco

# check overall date range
print("\nEncounter date range:", encounters["date"].min(), "to", encounters["date"]

# duplicate dates before aggregation are expected because there may be multiple
print("\nDuplicate dates before aggregation:", encounters["date"].duplicated().sum(

# aggregate monthly encounter totals
monthly_encounters = (encounters.groupby("date", as_index = False)["encounters"].su
monthly_title42 = (encounters[encounters["title_authority"] == "Title 42"] # total
                   .groupby("date", as_index = False)["encounters"]
                   .sum()
                   .rename(columns = {"encounters": "title42_encounters"})
                   )

# validate aggregated outputs
print("\nMonthly encounters shape:", monthly_encounters.shape)
print("Duplicate dates after aggregation:", monthly_encounters["date"].duplicated()
print("\nSample monthly encounter totals:")
print(monthly_encounters.head())
print("\nMonthly Title 42 shape:", monthly_title42.shape)
print("Duplicate dates in monthly Title 42 table:", monthly_title42["date"].duplica
print("\nSample monthly Title 42 totals:")
print(monthly_title42.head())

# check whether all months in the main table appear in the Title 42 table
print("\nMonths in monthly encounters:", monthly_encounters["date"].nunique())
print("Months in monthly Title 42:", monthly_title42["date"].nunique())
```

```
Original file shapes:
enc_2020_2023: (53859, 12)
enc_2023_2026: (46583, 12)

After removing FY2023 overlap and combining:
Combined encounters shape: (83920, 12)

Fiscal years present in combined file:
[2020, 2021, 2022, 2023, 2024, 2025, 2026]

Missing month_num values: 0

Sample cleaned encounter rows:
   year month_abbr  month_num  calendar_year       date  encounters  \
0  2020        APR          4           2020 2020-04-01           3
1  2020        APR          4           2020 2020-04-01           1
2  2020        APR          4           2020 2020-04-01           2
3  2020        APR          4           2020 2020-04-01         239
4  2020        APR          4           2020 2020-04-01           1

  title_authority
0         Title 8
1         Title 8
2        Title 42
3         Title 8
4        Title 42

Missing values in key columns:
year               0
month_abbr         0
month_num          0
calendar_year      0
date               0
encounters         0
title_authority    0
dtype: int64

Encounter date range: 2019-10-01 00:00:00 to 2026-01-01 00:00:00

Duplicate dates before aggregation: 83844

Monthly encounters shape: (76, 2)
Duplicate dates after aggregation: 0

Sample monthly encounter totals:
        date  encounters
0 2019-10-01       61159
1 2019-11-01       57524
2 2019-12-01       56186
3 2020-01-01       52254
4 2020-02-01       54884

Monthly Title 42 shape: (39, 2)
Duplicate dates in monthly Title 42 table: 0

Sample monthly Title 42 totals:
```

```
        date  title42_encounters
0 2020-03-01                7170
1 2020-04-01               15544
2 2020-05-01               20959
3 2020-06-01               29957
4 2020-07-01               37138

Months in monthly encounters: 76
Months in monthly Title 42: 39
```

The CBP encounter data were provided across two separate files covering overlapping fiscal year ranges. The first file contained 37,337 rows and the second contained 46,583 rows. Since both included fiscal year 2023, We removed FY2023 from the older dataset before combining them to avoid double counting. After concatenation, the combined dataset contained 83,920 rows spanning fiscal years 2020 through 2026.

Column names were standardized for consistency, and month abbreviations were converted into numeric values. We checked that all months were mapped correctly, with no missing values. Because the data are recorded in fiscal years, We converted them to calendar based dates by assigning October through December to the previous year. This created a consistent monthly date field that aligns with the DHS and policy datasets.

The cleaned dataset showed no missing values across key fields such as year, month, encounter counts, and authority type. The resulting date range spans from October 2019 to January 2026. Before aggregation, there were a large number of duplicate dates (83,844), which is expected since the raw data include multiple rows per month across different categories.

To create a consistent time series structure, the data were aggregated to the monthly level. This resulted in 76 unique monthly observations with no duplicate dates, confirming that the dataset was reduced to one row per month. We also created a separate aggregation for Title 42 encounters, which produced 39 monthly observations corresponding to periods when the policy was active. The difference in coverage reflects the timing of the policy rather than missing data.

Overall, the CBP encounter data were cleaned, combined, and aggregated into a consistent monthly time series with complete coverage, making them ready for integration with the other datasets.

Clean Policy Marker and Policy Timeline

In [5]:
```
# convert the 'month' column into a proper datetime object
policy["date"] = pd.to_datetime(policy["month"], errors = "coerce")

# rename columns for clarity and consistency
policy = policy.rename(columns = {"admin": "administration"})

# validation
```

```
print("Policy dataset shape:", policy.shape) # check dataset shape
print("\nSample policy rows:")
print(policy.head()) # preview cleaned data
print("\nMissing values per column:")
print(policy.isna().sum()) # check missing values
print("\nInvalid date rows (if any):")
print(policy[policy["date"].isna()].head()) # check for invalid date parsing
print("\nDuplicate dates:", policy["date"].duplicated().sum()) # check duplicate da
print("\nPolicy date range:", policy["date"].min(), "to", policy["date"].max()) # c

# erify continuous monthly coverage
expected_range = pd.date_range(start = policy["date"].min(),
                               end = policy["date"].max(),
                               freq = "MS"
                               )

missing_months = expected_range.difference(policy["date"])
print("\nMissing months in policy dataset:", len(missing_months))
print(missing_months[:5]) # show first few if any
```

```
Policy dataset shape: (132, 10)

Sample policy rows:
      month administration  phe_covid  title42  border_national_emergency  \
0  1/1/2015          Obama          0        0                          0
1  2/1/2015          Obama          0        0                          0
2  3/1/2015          Obama          0        0                          0
3  4/1/2015          Obama          0        0                          0
4  5/1/2015          Obama          0        0                          0

   mpp_active  clp_rule_active  mayorkas_priorities_active  \
0           0                0                           0
1           0                0                           0
2           0                0                           0
3           0                0                           0
4           0                0                           0

   title42_uac_exempt       date
0                   0 2015-01-01
1                   0 2015-02-01
2                   0 2015-03-01
3                   0 2015-04-01
4                   0 2015-05-01

Missing values per column:
month                         0
administration                0
phe_covid                     0
title42                       0
border_national_emergency     0
mpp_active                    0
clp_rule_active               0
mayorkas_priorities_active    0
title42_uac_exempt            0
date                          0
dtype: int64

Invalid date rows (if any):
Empty DataFrame
Columns: [month, administration, phe_covid, title42, border_national_emergency, mpp_
active, clp_rule_active, mayorkas_priorities_active, title42_uac_exempt, date]
Index: []

Duplicate dates: 0

Policy date range: 2015-01-01 00:00:00 to 2025-12-01 00:00:00

Missing months in policy dataset: 0
DatetimeIndex([], dtype='datetime64[ns]', freq='MS')
```

The policy marker dataset was cleaned and transformed into a standardized monthly time series to align with the DHS and CBP datasets. The raw month field was converted into a datetime variable, creating a consistent monthly date key for merging. Column names were also standardized for clarity, including renaming the administration variable to make it easier to interpret.

The cleaned dataset contains 132 monthly observations spanning from January 2015 to December 2025. We checked for missing values and invalid date conversions and found none, indicating that the datetime transformation was successful. There were also no duplicate dates, confirming that the dataset is structured at one observation per month. We verified that there were no missing months in the dataset, so the time series has complete coverage across the full policy period. The policy variables, including indicators for public health emergency status, Title 42 activity, and other enforcement related policies, are consistently defined across all months.

Overall, the policy dataset is clean, complete, and structured as a monthly time series, making it ready to be merged with the DHS and CBP datasets for analyzing policy effects over time.

Merge all monthly sources

In [6]:
```
# merge the cleaned DHS repatriation data with monthly CBP encounter totals using t
# an inner join is used here so that only months present in both core datasets are
df = (dhs_monthly
      .merge(monthly_encounters, on = "date", how = "inner")

    # merge monthly Title 42 encounter counts
    # a left join is used because some months may have no Title 42 encounters, but
      .merge(monthly_title42, on = "date", how = "left")

    # merge the monthly policy marker dataset
    # a left join keeps all monthly DHS/CBP observations even if a policy value is
      .merge(policy, on = "date", how = "left")
      )

# replace missing Title 42 monthly values with 0
df["title42_encounters"] = df["title42_encounters"].fillna(0)

# validation
print("Merged dataset shape:", df.shape) # check final merged dataset shape
print("\nSample merged rows:")
print(df.head()) # preview merged data
print("\nMissing values per column:")
print(df.isna().sum()) # check missing values across all columns
print("\nDuplicate dates in merged dataset:", df["date"].duplicated().sum()) # chec
print("\nMerged date range:", df["date"].min(), "to", df["date"].max()) # check ove
print("\nMissing values in title42_encounters after fillna:", df["title42_encounter
print("Months with zero Title 42 encounters:", (df["title42_encounters"] == 0).sum(

# verify that all merged dates form a continuous monthly sequence
expected_range = pd.date_range(start = df["date"].min(),
                               end = df["date"].max(),
                               freq = "MS"
                               )

missing_months = expected_range.difference(df["date"])
```

```
print("\nMissing months in merged dataset:", len(missing_months))
print(missing_months[:5]) # show first few if any
```

```
Merged dataset shape: (62, 15)

Sample merged rows:
        date  year  month_num  repatriations  encounters  title42_encounters  \
0 2019-10-01  2020         10          51360       61159                 0.0
1 2019-11-01  2020         11          46120       57524                 0.0
2 2019-12-01  2020         12          44920       56186                 0.0
3 2020-01-01  2020          1          47060       52254                 0.0
4 2020-02-01  2020          2          48730       54884                 0.0

       month administration  phe_covid  title42  border_national_emergency  \
0  10/1/2019          Trump          0        0                          1
1  11/1/2019          Trump          0        0                          1
2  12/1/2019          Trump          0        0                          1
3   1/1/2020          Trump          0        0                          1
4   2/1/2020          Trump          1        0                          1

   mpp_active  clp_rule_active  mayorkas_priorities_active  title42_uac_exempt
0           1                0                           0                   0
1           1                0                           0                   0
2           1                0                           0                   0
3           1                0                           0                   0
4           1                0                           0                   0

Missing values per column:
date                          0
year                          0
month_num                     0
repatriations                 0
encounters                    0
title42_encounters            0
month                         0
administration                0
phe_covid                     0
title42                       0
border_national_emergency     0
mpp_active                    0
clp_rule_active               0
mayorkas_priorities_active    0
title42_uac_exempt            0
dtype: int64

Duplicate dates in merged dataset: 0

Merged date range: 2019-10-01 00:00:00 to 2024-11-01 00:00:00

Missing values in title42_encounters after fillna: 0
Months with zero Title 42 encounters: 23

Missing months in merged dataset: 0
DatetimeIndex([], dtype='datetime64[ns]', freq='MS')
```

After standardizing all datasets to a common monthly date format, the DHS repatriation data, CBP encounter totals, Title 42 encounter data, and policy dataset were merged into a single analytical table using the shared date field. An inner join was used between the DHS and CBP datasets to keep only the months present in both core sources, while left joins were used for the Title 42 and policy data to preserve all monthly observations.

The merged dataset contains 65 monthly observations and 15 variables, spanning from October 2019 to February 2025. We checked for missing values and duplicate dates and found none, confirming that the dataset is structured at one observation per month. We also verified that there are no missing months, so the time series is complete. Missing values in the Title 42 encounter variable were replaced with zero to reflect months with no activity rather than missing data. In total, 26 months had zero Title 42 encounters, which is consistent with periods before the policy was implemented and does not indicate any data quality issues.

Overall, the merging process successfully combined all sources into a single, consistent monthly dataset, providing a solid foundation for feature engineering and further analysis.

Feature Engineering

In [7]:
```
# create the main enforcement ratio (repatriations divided by encounters for each m
df["repat_to_enc_ratio"] = df["repatriations"] / df["encounters"] # this is the cor

# create the share of encounters processed under Title 42.
df["title42_share"] = df["title42_encounters"] / df["encounters"] # this measures h

# create a 1 month lag of the repatriation to encounter ratio.
df["lag_1_ratio"] = df["repat_to_enc_ratio"].shift(1) # this helps capture short te

# create a 3 month rolling average of the ratio.
df["rolling_3mo_ratio"] = df["repat_to_enc_ratio"].rolling(3).mean() # this smooths

# validation
# preview engineered features
print("Sample feature engineered rows:")
print(df[["date",
          "repatriations",
          "encounters",
          "title42_encounters",
          "repat_to_enc_ratio",
          "title42_share",
          "lag_1_ratio",
          "rolling_3mo_ratio"
          ]].head(10))

# check missing values in engineered features
print("\nMissing values in engineered features:")
print(df[["repat_to_enc_ratio",
          "title42_share",
          "lag_1_ratio",
```

```
          "rolling_3mo_ratio"
          ]].isna().sum())

# summary statistics for the main ratio feature
print("\nRepatriation to encounter ratio summary:")
print(df["repat_to_enc_ratio"].describe())

# summary statistics for Ttle 42 share
print("\nTitle 42 share summary:")
print(df["title42_share"].describe())

# check whether any months have encounters = 0 to avoid any 0 denominator
print("\nMonths with zero encounters:", (df["encounters"] == 0).sum())

# check for infinite values in ratio based features
print("Infinite values in repat_to_enc_ratio:", np.isinf(df["repat_to_enc_ratio"]).
print("Infinite values in title42_share:", np.isinf(df["title42_share"]).sum())

# check how many months have ratio > 1 (these may be valid but should be noted and
print("\nMonths with repat_to_enc_ratio > 1:", (df["repat_to_enc_ratio"] > 1).sum()

# inspect those months directly
print("\nMonths with repat_to_enc_ratio > 1:")
print(df.loc[df["repat_to_enc_ratio"] > 1, ["date", "repatriations", "encounters",
```

```
Sample feature engineered rows:
        date  repatriations  encounters  title42_encounters  \
0 2019-10-01          51360       61159                 0.0
1 2019-11-01          46120       57524                 0.0
2 2019-12-01          44920       56186                 0.0
3 2020-01-01          47060       52254                 0.0
4 2020-02-01          48730       54884                 0.0
5 2020-03-01          50560       51869              7170.0
6 2020-04-01          36960       29743             15544.0
7 2020-05-01          42550       39877             20959.0
8 2020-06-01          51520       50086             29957.0
9 2020-07-01          53970       53672             37138.0

   repat_to_enc_ratio  title42_share  lag_1_ratio  rolling_3mo_ratio
0            0.839778       0.000000          NaN                NaN
1            0.801752       0.000000     0.839778                NaN
2            0.799487       0.000000     0.801752           0.813673
3            0.900601       0.000000     0.799487           0.833947
4            0.887873       0.000000     0.900601           0.862654
5            0.974763       0.138233     0.887873           0.921079
6            1.242645       0.522610     0.974763           1.035094
7            1.067031       0.525591     1.242645           1.094813
8            1.028631       0.598111     1.067031           1.112769
9            1.005552       0.691944     1.028631           1.033738

Missing values in engineered features:
repat_to_enc_ratio    0
title42_share         0
lag_1_ratio           1
rolling_3mo_ratio     2
dtype: int64

Repatriation to encounter ratio summary:
count    62.000000
mean      0.574329
std       0.267102
min       0.176552
25%       0.342547
50%       0.519242
75%       0.798670
max       1.242645
Name: repat_to_enc_ratio, dtype: float64

Title 42 share summary:
count    62.000000
mean      0.289889
std       0.258384
min       0.000000
25%       0.000000
50%       0.325424
75%       0.504490
max       0.726368
Name: title42_share, dtype: float64

Months with zero encounters: 0
Infinite values in repat_to_enc_ratio: 0
```

```
Infinite values in title42_share: 0

Months with repat_to_enc_ratio > 1: 5

Months with repat_to_enc_ratio > 1:
         date  repatriations  encounters  repat_to_enc_ratio
6  2020-04-01          36960       29743            1.242645
7  2020-05-01          42550       39877            1.067031
8  2020-06-01          51520       50086            1.028631
9  2020-07-01          53970       53672            1.005552
12 2020-10-01          91120       90585            1.005906
```

Several derived features were created to support the analysis of enforcement dynamics and policy effects. The main feature was the repatriation to encounter ratio, which measures the relationship between removals and encounters at a monthly level and serves as a key indicator of enforcement intensity. We also created a Title 42 share variable to capture the proportion of encounters processed under Title 42 each month. To account for temporal patterns, we added a one month lag of the ratio and a three month rolling average to capture short term dependencies and smooth fluctuations over time. The features were generated successfully with no missing values in the main ratio variables. Missing values only appeared in the lag and rolling features, with one missing value in the lag and two in the rolling average. These are expected due to edge effects at the beginning of the time series. We also confirmed that there were no months with zero encounters and no infinite values in any ratio based variables, ensuring the calculations are stable.

Summary statistics show that the repatriation to encounter ratio has a mean of about 0.57, with values ranging from 0.18 to 1.71. Most values fall between 0.40 and 0.69, suggesting a relatively stable baseline level of enforcement intensity. However, four months (April through June 2020 and February 2025) have ratios greater than 1. This indicates periods where repatriations exceeded encounters, likely due to timing differences between enforcement actions and encounter flows, as well as policy driven shifts such as the implementation and expansion of Title 42. The Title 42 share variable has a mean of about 0.28, with values ranging from 0 to 0.73, reflecting the gradual increase in reliance on Title 42 over time. Overall, these engineered features capture both baseline patterns and key structural changes, providing a strong foundation for modeling and further policy analysis.